

Hnnu 队伍策略说明书

FIRA SimuroSot 5v5 仿真组 · 2026-09-14 · 本文写给"完全没接触过这个平台的人"

一分钟看懂我们：我们写一个 Strategy4Blue.dll（以及同源的 Strategy4Yellow.dll），交给平台加载。平台每 1/40 秒（25 毫秒）问我们一次"你队 5 台机器人左右轮各转多快"，我们回答一次。

我们的打法一句话：能射就射 → 射不了就传 → 被围就脱身 → 丢球立刻分层回防，而**所有战术的前提**是**"绝不犯规"**（这个平台判罚极重，见第 5 节）。

我们的强项：纪律 + 工程验证体系（每个改动先跑单元测试 → 再跑 50 场无头仿真 → 再上真机）。

1 · 平台长什么样（配界面截图）

比赛跑在两个窗口里：左边是**球场窗口**（WorldModel），右边是我们策略被调到点球点的瞬间；另一个是**主对话框**，显示剩余时间和比分。

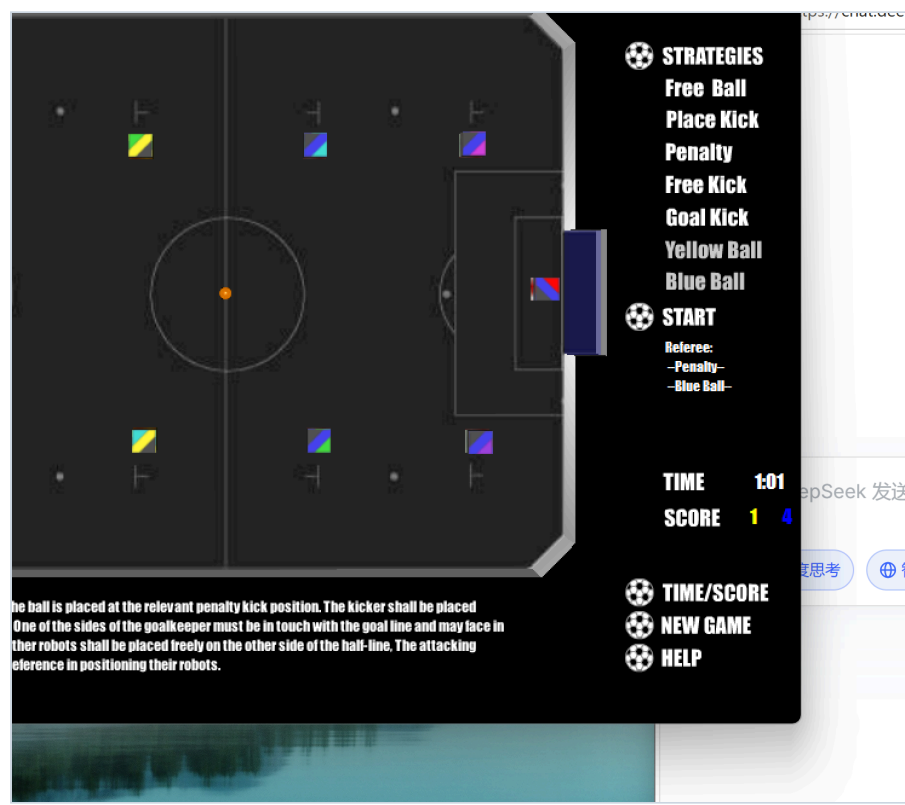


图 1 · 平台球场窗口（WorldModel 的界面）：左边是 220×180 厘米的球场，我们我方 5 台机器人（黄三角/蓝方块）与球都在这里；右边是策略菜单与 START、TIME/SCORE、HELP（HELP 里就是平台自带的规则原文）。

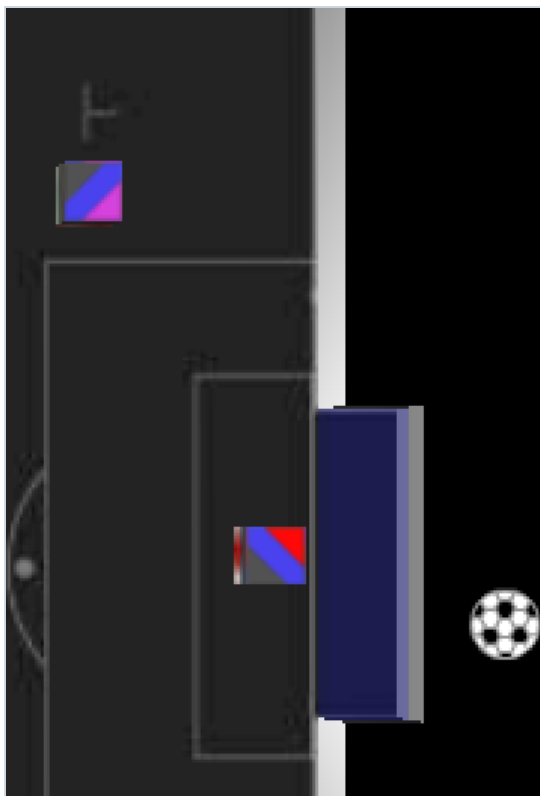


图 2 · 平台主对话框（SimuroSot5）：显示剩余时间与比分。比分格式是（黄队：蓝队）。

读图要点：球场左上到右下是 220×180 厘米；每队 5 台机器人； **比分格式是（黄队：蓝队）**——所以看到（2：4）是我们赢了 4:2。

2 · 我们每帧做的三件事

顺序	做什么	输出
① 读世界	把平台给的 5 台自机、5 台敌机、球（含当前/上一帧/平台预测三份）读进我们的"世界模型"	WorldModel（全队共享的黑板）
② 判局势	算球权、威胁等级、攻防状态（带 3 帧防抖），并算出 3 个站位参考点	球权 / 威胁 / 战斗姿态
③ 派活	5 个角色各跑一次自己的决策，写出左右轮速	每台机器人的 <code>v1 / vr</code>

角色是固定的（不是每帧抢活）：0 号=守门员、1 号=主攻、2 号=助攻、3 号=中场、4 号=后卫（清道夫）。

3 · 12 种比赛状态与摆位

平台把比赛分成 12 种状态（正常比赛 / 争球 / 开球 / 点球 / 任意球 / 门球，每种"黄/蓝"代表**谁主罚**）。每种状态开始时，平台会要求双方先摆好机器人——这一层由我们的 `formation.cpp` 负责：

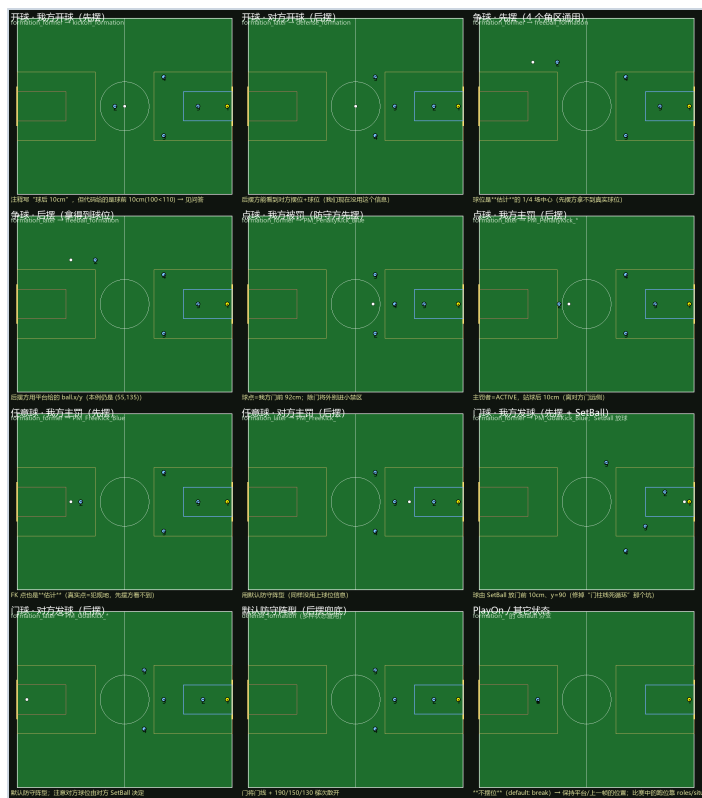


图 3·12 种比赛状态的摆位示意（按蓝队坐标画，黄队镜像）：开球 / 争球 / 点球 / 任意球 / 门球，以及每种状态里"主罚方先摆、防守方后摆"的差别。

摆位层我们自己的设计：坐标一律按蓝队写，再用"镜像"公式换算到黄队（ $M(x) = \text{蓝} ? x : 220-x$ ） $\Rightarrow$ **同一份代码两侧通用**，这也是"比赛万一分到黄队"能一键切换的原因。另外两条来自真机复盘的修正：**点球时踢球人必须站在球后、开球时人必须在本方半场**。

4·九条核心算法（每条：一句话 → 怎么做 → 为什么）

4.1 球速跳变滤波 —— 别被“瞬移的球”骗了

一句话：平台在进球/摆位时会把球瞬移（实测一帧跳 117 厘米），直接差分算速度会得到物理上不可能的巨大速度，所以超过 30 厘米/帧就把这一帧的速度记成 0。

```
ball.vx = (|本帧x - 上帧x| > 30) ? 0 : 差分值; // 30cm/帧≈1200cm/s，真实球速只到 ~4cm/帧
```

为什么阈值是 30：正常球速最高约 22 厘米/帧、复位跳变 100+，30 卡在中间。曾经取 12 → 真机被打 0:3（真球速被误当跳变清零，主攻以为球没动、站着不追 X）。

4.2 球权与威胁分级 —— 决定全队是“压上”还是“回收”

一句话：谁离球更近、并且近到 20 厘米内，就算谁控球；球在我们禁区 → 威胁 1.0，在我们半场 → 0.6，我们控球 → 0.1。

```
we_have_ball = (我方最近者到球 < 对方最近者到球) && (我方最近者 < 20cm);
```

为什么不用“球在谁半场”：对方压上进攻时球就在我们半场，但那是**对方的球**——按半场判会让全队误切进攻态、后防空虚。另外平台也给一个球权字段，我们做成了分层使用：局面明确时按距离算，散球混战时才听平台的。

4.3 站位锚点 + 15 厘米滞回 —— 让机器人跑直线

一句话：助攻/中场该站哪，每帧都在变（跟着球变）；但我们只在“新目标离旧锚点 ≥ 15 厘米”时才真的改目标，否则就让它按原目标跑直线。

```
if (dist(新目标, 记录的锚点) >= 15.0) { 记录锚点 = 新目标; } // 差不够就不动
```

为什么：差速轮不能横着走，改方向必须先转身——目标每帧挪几厘米，机器人就每帧微转一下，走成 S 形、永远追不上、也永远到不了位。15 厘米这道闸门让路径变成“几段长直线”。**防守锚点故意不设滞回**（防线上差 15 厘米就是一个空档）。

4.4 射门：先看“门将挡住了多少”，再决定瞄哪

一句话：把球门看成一个扇面，门将挡住中间一块，我们瞄**剩下最宽那段空隙的正中**。

```
门张角 ±half（球到两门柱的夹角）；  
门将遮挡 [gk_mid-gk_half, gk_mid+gk_half]（半径 8cm 在该距离的张角）；  
净开口 = 门张角扣除遮挡后的最大连续空隙 → 瞄它的中心。
```

再加三道闸门： **≤ 70 厘米无条件射**（仿真+真机都验证过的主力区）；**70~110 厘米**要求净开口 $\geq 8^\circ$ 、球前 30 厘米无人挡、且机会质量 ≥ 0.35 ；**点球**走旁路（白送的机会不能被闸门拒掉）。

机会质量（0~1，越大越该射）= $0.5 \times \text{开口分} + 0.3 \times \text{距离分} + 0.2 \times \text{球速分}$ 。

4.5 差速轮控制：让机器人“恰好停住”

一句话：命令速度不超过 $\sqrt{2 \cdot \text{加速度} \cdot \text{剩余距离}}$ ——这是数学上“从当前位置恰好刹车停在目标点”的临界曲线。

```
double drive = sqrt(2.0 * kBrakeAccel * max(0.0, de - kStopEps)); // 时间最优制动包线
if (drive > v_max) drive = v_max;
```

这就是双积分器时间最优控制里的"切换曲线"（bang-bang 的边界）。改之前速度随距离用 sigmoid 控制，离目标 5 厘米就命令满速、必然冲过头 → 倒车 → 极限环（实测低速来回蹭 2.3 次/分钟）。顺带一个已验证的教训：**把整条速度律都换成包线，仿真会崩**（净胜 +0.3→-2.4），所以只在"到点停"这类任务上用它。

4.6 避障路径：4 厘米网格 A* + 视线拉直

一句话：把球场切成 4 厘米的格子，用 A*（带方向启发的图搜索）找一条不撞人的路，再把阶梯状路径拉直成 2~4 个折点。

- 直线能走就直接走（绝大多数时候）；
- "整格撞到人"的格子判为不可走 ⇒ 网格路径严格不穿人；
- 边线 4 厘米内也当墙（旧算法完全没有边界概念，路径可以贴线甚至出界）；
- 实测：绕行长度离理论最优只差 **0.9%**，200 组随机场景比旧算法多解 7 组，耗时同量级（~20 微秒/次）。

4.7 守门员：预测、封线、不要碰球

机制	做法	真机教训
预测入球点	按球速外推到门线，得到"球会从哪个 y 进门"，再按到线时间决定出击深度	球贴边墙滚时直线外推会算错 ⇒ 加了 撞墙反射 版预测
门线封堵	球已经在门框内的轨迹上 → 直接抢门线上的预测落点（优先于清球/站位分支）	复盘两个丢球：门将当时在追"球侧方 15 厘米的绕行点"，球从旁边进
球外侧禁推	球已越过自己（比门将更靠门）时，先横向让开， 绝不朝球推进	逐帧证据：两个乌龙球的球速方向在"门将碰到球"那一帧翻转成进门
清球先转正	门前死球要推出去时，机头没对准就先原地转正，再直线推穿	曾出现门球卡 7.6 秒 球一动不动（平台每 5 秒重发一次）

4.8 定位球：先"认出"这是我们的点球

一句话：平台在执行期**不会**把"点球"这个状态告诉我们（实测 16000 帧里"正常比赛"一帧都没出现），所以我们改用可观测量：**球静止在对方罚球点上**（实测点 = 门前 39.4 厘米、正中央，两场 14 次摆球完全一致）。

```
we_take_penalty = 球静止(<1cm/帧) && |球x - (对方门线 ± 39.4)| < 1.5 && |球y - 90| < 1.5;
```

认出之后：踢球人就地转正、**立刻推**（真机实测：如果先倒车助跑 0.6 秒，对手会从 34 厘米逼近到 10 厘米把球截走）。

4.9 防守工具箱

工具	机制	关键参数
区域断球点	球轨迹 ∩ 门前拦截线；球朝边墙滚时用 反射 后的轨迹	拦截线 = 门前 50 厘米
人盯人	给 5 个对手打威胁分（离球远近 + 离门远近 + 球是否正传给他 + 是否持球突破），取最高分者贴身	贴身 16 厘米；预测外推 3 帧；5% 换人滞后；离球/离门 >40 厘米不值得贴

二抢一	持球者压到门前时，再派一个防守者形成包夹；只派"最近的那一个"，另一个留区域保纵深	离门 <100 厘米才夹；球在罚球区内要持球者 离门 <45 厘米
抢反弹位	对方射门时，两人分别站在罚球区前缘的左右两侧等门将扑出的第二落点	只在球朝门速 >8 厘米/顿时才抢

5 • 为什么"别犯规"比战术更值钱

犯规	代价 (规则原文)	我们的实测
对方门区内 2+ 人 / 单人停留 >20 周期	判对方点球	纪律修复前我们一场送 12 次点球 X, 修复后 3~6 次
禁止推球区 (四角黄区) 推球	每 4 次给对方 +1 球 + 判争球	实测 4 次 = 白送 1 球
僵局 (门区外 100 周期无进展)	判争球	我们的一场争球重置曾达 100~110 次

所以我们把纪律做成了代码里的硬约束：门区停留计数器（纯停留 8 帧、总时长 20 帧就撤出）、角区不许推球、死球期不碰球（但保留"球静止 100 帧还没人来处理就主动去推"的超时逃生，防死球变僵局）。

6 • 我们怎么验证（这是我们认为最值钱的部分）

- **单元测试**：offline_test.exe 覆盖 186 个函数；规矩是"先让测试失败，再改代码"（例如给门将新分支时，故意关掉它让测试报 FAIL，证明测试真的能抓住问题）。
- **无头仿真**：sim_bench 不开窗口跑完整比赛，50 场约 20 秒，自动出净胜球/射门/纪律对照表。
- **真机黑匣子**：临时变体 DLL 把平台每一帧的原始字段写成 CSV（游戏状态、球权、球速...），用来回答"文档里查不到、只能实测"的问题（例如发现平台在执行期根本不报"点球"状态）。
- **逐帧复盘**：.rlg 录像逐帧 + 平台官方日志（事件日记）交叉验证；进球/丢球/判罚都能定位到帧号。

一个真实案例（点球 0/9 的根因）：官方规则速查里把状态名写成"黄/蓝被**罚**"，而平台实际语义是"黄/蓝**主罚**"。我们通过三条独立证据（官方 demo 的摆位回调、日志比分归因、录像里球被摆在哪个罚球点）确认后修正了代码和文档——此前 9 次点球"一次都没踢"就是这一个字造成的。

7 • 目前战绩与已知短板

项目	现状
近期战绩 (对官方 demo)	近 6 场 3 胜 1 平 2 负 (5:3、1:0、4:2、1:1 0:2、0:1)
点球转化率	约 11% (18 次 2 球) —— 最大得分空间，也是当前重点
丢球性质	最近两场 3 个丢球 全部疑似乌龙 (球速方向证据：门将碰球那一帧球转向自家门)
已知短板	罚点球时瞄准转正白耗 1.12 秒；机会质量还没算"门将够不够得到"这个时间维度
已修复	门球卡死 7.6 秒 → 转身即推；点球识别 18/18 成功；定位球重发 0 次

8 • 附录：模块分工与参数速查

模块	文件	职责
----	----	----

世界模型 / 队伍上下文	world_model.cpp team.hpp	平台状态 → 内部结构；球速滤波；坐标系与镜像
局势与站位	situation.cpp strategy.cpp	球权/威胁/攻防状态机/三个站位锚点
角色调度	roles.cpp role_assignment.cpp	门将/主攻/助攻/中场/后卫的决策与调度
射门 / 传球	shoot.cpp pass.cpp	净开口角、机会质量、传球点选择
防守工具箱	defense.cpp/hpp	断球点（含反射）、人盯人、二抢一、抢反弹位
运动与路径	motion.cpp route.cpp	制动包线、到点定向、4 厘米网格 A*
定位球摆位	formation.cpp	12 态摆位、摆球

参数	值	含义
射程闸门	$\leq 70\text{cm}$ 无条件 / $70\sim 110\text{cm}$ 需净开口 $\geq 8^\circ$	超过 110 厘米不射，改带球或传球
机会质量门限	≥ 0.35 （仅远射档判）	$0.5\times\text{开口} + 0.3\times\text{距离} + 0.2\times\text{球速}$
到点速度律	$v = \min(150, \sqrt{(2\cdot 400\cdot (s-1.5))})$	时间最优制动包线（cm、cm/s）
避障网格	4 厘米 / 55×45 格	A* + 视线拉直
锚点滞回	15 厘米	防守锚点不设
防守贴身	16 厘米、外推 3 帧	人盯人
罚球点	门前 39.4 厘米、正中央	实测值，用于识别我方点球

本文由 tools/py/make_strategy_pdf.py 自动生成；截图来自真机录像窗口（build/penalty_shots/），12 态摆位图由 tools/py/formation_diagram.py 生成。所有数据均来自真机录像（.rlg）与平台官方日志的逐帧复盘。